

Oracle Forms → Migration Strategy

Assumptions: internal-only use (but at least network/SSO-level access control and an audit log); read-only functionality (search, view — no create/update indicated); two cutover scenarios to cover (parallel run vs. single-version).

a) Proposed Architecture

Classic 3-tier, with the database keeping its role as the system of record **and** business-rule owner:

```
Vue.js SPA → Java REST API (Spring Boot) → Oracle DB (PL/SQL packages, views)
```

- **Frontend:** Vue 3 SPA, calls backend only via REST/JSON. No direct DB access.
- **Backend:** Spring Boot service, stateless, thin orchestration layer. Calls existing (or newly organized) PL/SQL packages via stored procedure/function calls.
- **Database:** Oracle stays the authority for business rules, calculations, and complex joins (search/filter logic, invoice/payment aggregation).

b) Frontend Structure (Vue.js)

- Vue 3 + Composition API + TypeScript, Vite build.
- Views map 1:1 to existing Forms canvases for a smooth transition.
- **Pinia** store holds "selected customer" context, replacing Forms' shared block/global variables so navigating between tabs keeps context, as in Forms.
- Data grid component (e.g., PrimeVue/AG Grid) to replicate Forms' multi-record block behavior (sorting, paging, inline filters).
- Axios service layer with interceptors for error handling and (future) auth token injection.
- Keyboard-navigation support is worth prioritizing — Forms power-users rely heavily on it, and losing it is a common source of user pushback in these migrations.

c) Backend Integration Approach

- Spring Boot REST API: Controller → Service → DAO layers.
- DAO layer calls PL/SQL — not raw SQL against base tables — so validation, security predicates, and calculations already encoded in the DB are reused rather than reimplemented in Java.
- REF CURSOR results mapped to DTOs → JSON.
- Pagination handled at the SQL/PLSQL level (not fetching full result sets), since Forms' array-fetch tuning won't exist anymore.

d) Migration Approach

Phase 1 — Inventory: Extract all PL/SQL logic currently embedded in Forms triggers (WHEN-VALIDATE-ITEM, KEY-*, format masks, dynamic LOVs). Anything not already in a DB package

needs to be pulled out into one — this is usually the single biggest source of underestimated effort.

Phase 2 — API build: Build REST endpoints matching existing Forms blocks feature-for-feature. Resist redesigning behavior at this stage — get functional parity first, improve UX later.

Phase 3 — Frontend build & internal UAT: Vue app consuming the new API, tested against Forms output side-by-side (same searches, same customer, compare results/totals).

Phase 4 — Cutover, per two scenarios:

- **Scenario A — Parallel run (recommended if feasible):** Since this functionality is read-only, both Forms and the new app can point at the same DB simultaneously with no write-conflict risk. Roll out to a pilot group, compare usage, keep Forms as fallback, then decommission once confidence is high. This is the low-risk path precisely *because* there's no data mutation here — it would be materially riskier if the screen also allowed edits.
- **Scenario B — Single-version cutover:** Requires a full regression pass beforehand, a defined cutover window, and a rollback plan (re-enable Forms access) if critical defects surface in the first days. Communicate the freeze clearly since users lose the fallback immediately.

e) Stays in Oracle DB / PL/SQL

- All business logic and calculations (search filters, invoice totals, payment status derivation, any aggregate/statistical logic).
- Data access via stored procedures/functions; reporting/read views for complex joins.
- Referential integrity, triggers, sequences, VPD/row-level security if present — unaffected by the migration.

f) Moves Outside the DB

- Presentation/formatting logic (currently in Forms format masks, item triggers) → Vue components.
- Screen/navigation flow control (Forms PL/SQL navigation logic) → Vue Router.
- Session/shared-state handling (Forms global variables) → Pinia store client-side, or backend session/service state.
- Immediate field-level UI validation → Vue-side validation (keeping the DB as the final authority).
- Any OS-level calls or Forms-specific integrations (e.g., `HOST ( )` calls, `client_ole2`) → reimplemented in the Java layer if still needed.

g) Main Risks & Challenges

- **Hidden logic in Forms triggers** not visible without full source review — biggest risk to underestimate.
- **Dynamic search behavior** (Forms "Enter Query" mode, dynamic LOVs) is harder to replicate 1:1 in a REST API — needs a well-designed generic search endpoint.